

Redigering i fält utan mobiltäckning

Hur man gör Origo till en fältinventeringsapplikation som inte
behöver nätverkstäckning

(Inte hur man gör en Origoapplikation som backup när nätverket
kraschat)

Användningsfallet

- Verksamheten behöver en enkel "Collector"-app för att inventera företeelser i terränglådan
- Det skall finnas en detaljerad bakgrundskarta för att navigera sig
 - Ortofoto
 - Topowebb
- Det skall finnas ett redigerbart lager med möjlighet att spara ändringar till central databas
- Appen måste fungera utan nätverkstäckning

Problemet

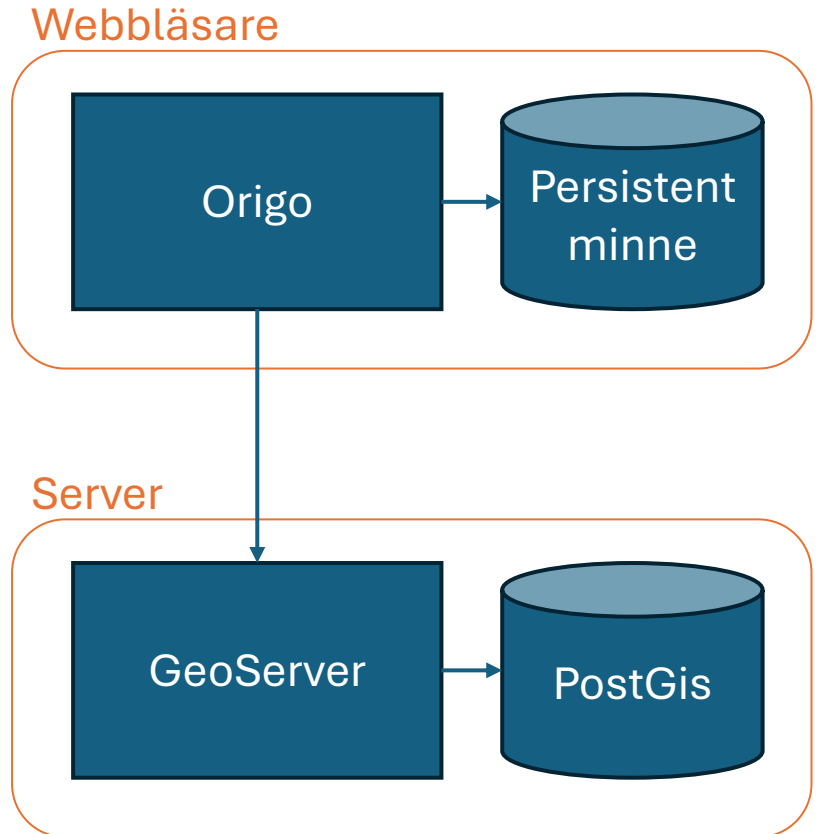
- Utan nätverkstäckning klarar inte Origo av att
 - Starta om appen om man navigerar bort webbläsaren
 - Strukturerat cacha rasterbakgrundskartor,
 - Har man tur så kan tiles cachas i webbläsaren där man redan visat kartan
 - Komma ihåg osparade redigeringar om man navigerar bort från appen
 - Vektorlager töms mellan varje session
 - Den befintliga editorns editstore töms mellan varje session
- Origo har haft en lageryp för offlineredigering, men den koden har degenererat då ingen använde den. Troligtvis för att det inte fanns stöd för bakgrundskartor vilket begränsade användningsområdet

Alternativen

- Implementera stöd i Origo
 - Igenkännande gränssnitt
 - Infrastrukturen på plats
- Köra tredjepartapplikation, e.x. Qfield
 - Högre inlärningströskel
 - Support på tredjepartsapplikation
 - Behov av infrastruktur för synkronisering
- Lokal kartserver som installeras på enheten
 - För PC skulle man kanske kunna packa ihop en standardprodukt, t.ex. Geoserver + Geopackage
 - För telefoner kanske man måste utveckla en enkel WFS + Tileserver
 - Kräver infrastruktur för att synkronisera datakatalogen
- Utveckla en egen native app
 - Webbapplikationer har inte med lätthet tillgång till filsystemet, men en native app skulle kunna jobba mot lokala filer
 - Behov av infrastruktur för synkronisering
 - Kul, men knappast kostnadseffektivt

Lösningen

- Implementera fem nya förmågor i Origo
 - Nytt lokalt rasterlager. Förladdar tiles från WMS till persistent minne på enheten
 - Nytt lokalt vektorlager. Förladdar vektorlager från WFS till persistent minne på enheten
 - Spara redigeringar i persistent minne på enheten
 - Möjlighet att synkronisera redigeringar mot server via WFS-T
 - Själva applikationen sparas i persistent minne när den väl laddats ner en gång
- Observera: De lokala lagren har ingen magisk förmåga att hantera omväxlande online/offline. De har alltid enhetens persistenta minne som källa så "lokala lager" är en mer rättvisande benämning än "offlinelager"
- Ingen förändring av infrastruktur
 - Allt går via standard GeoServer eller valfri OGC kartserver
- Än så länge ligger det som en PR i Origo



Starta applikationen offline

- Applikationen är en vanlig webbsida och startas med samma adress oavsett om man har cachat ner den eller ej
- Första gången man navigerar till sidan laddas alla filer som applikationen är beroende av ner i persistent minne i webbläsaren
- Vid efterföljande uppstarter startas den lokalt och är därför inte beroende av uppkoppling, men en bakgrundsprocess kontrollerar om det finns en ny version
- Mekanismen bakom detta är ServiceWorkers
- Man måste skapa en egen ServiceWorker för varje applikation beroende på vilka filer man behöver och hur man vill att applikationen skall bete sig
- Det finns en mekanism i origo för att initiera en ServiceWorker, men ingen färdig ServiceWorker
- Det finns ett exempel på ServiceWorker i Origo repositoryt



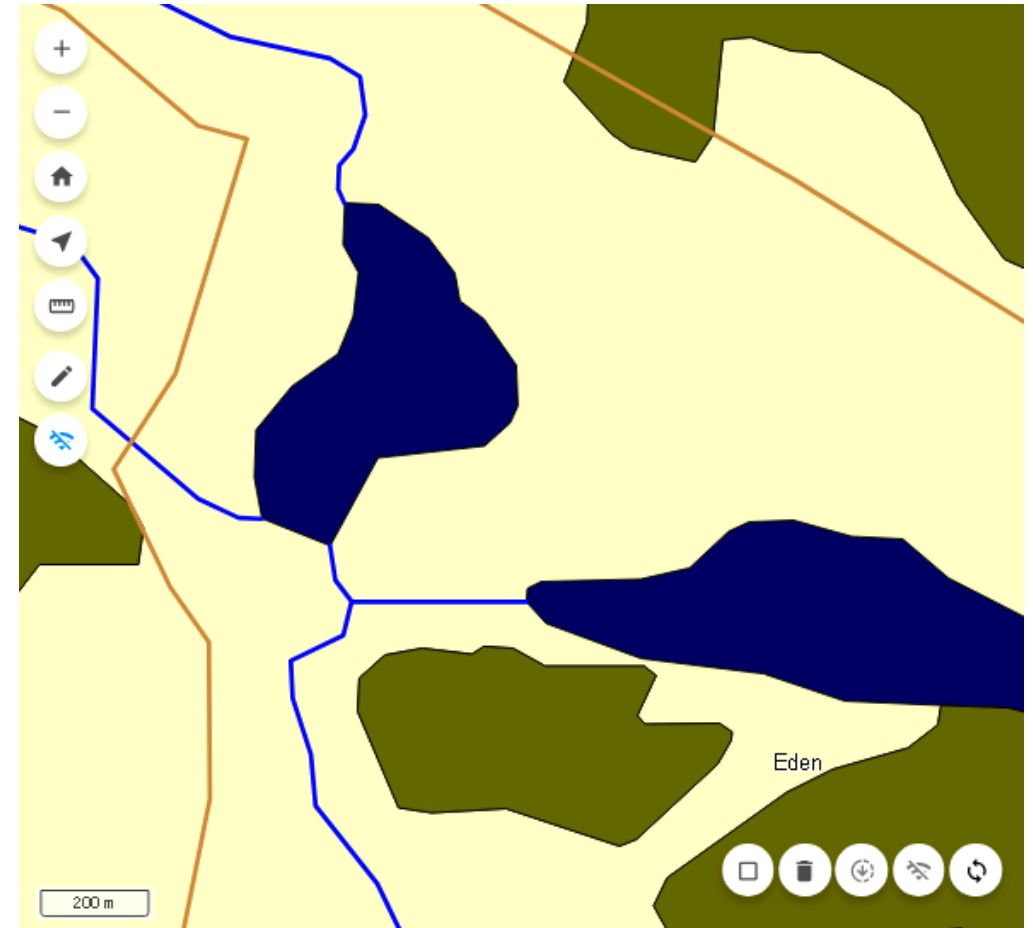
Bakgrundskarta

- För att navigera sig över en större yta kan man ha en bakgrundskarta
- Ingår i applikationens ServiceWorker som GeoJson-filer.
 - påverkar applikationens nedladdningstorlek och kräver en ny version vid uppdatering av bakgrundskartan
- Man kan också välja att ha WFS-lager som man synkar över som ett lokalt vektorlager
 - Påverkar varken appens nedladdningstorlek eller version, men kräver användarinteraktion innan den kan användas



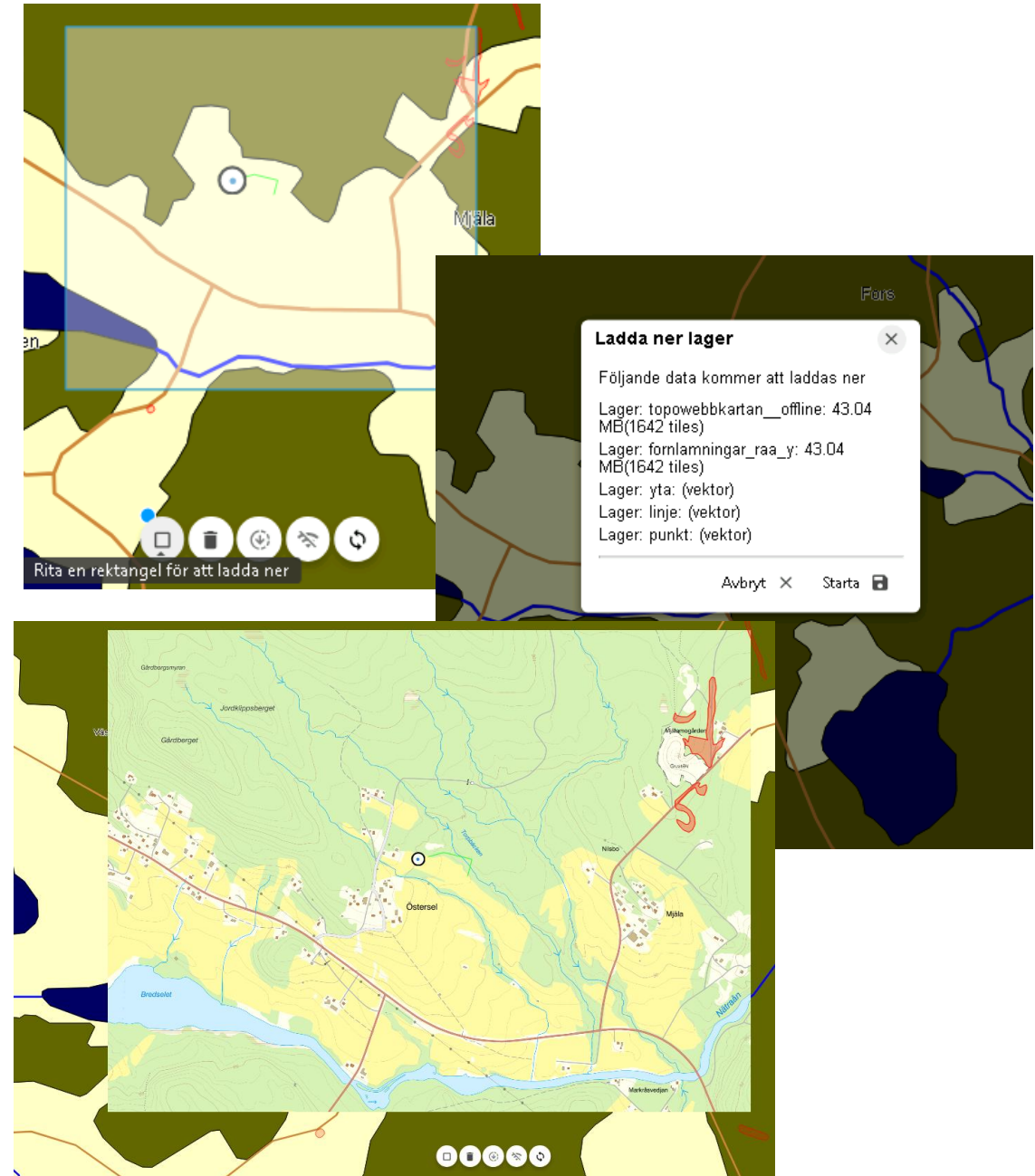
Offlineverktygslådan

- Fem verktyg i en verktygslåda
 - Välj område att ladda ner
 - Rensa ledladdade områden
 - Visa pågående nedladdningar
 - Ändra till offlineläge
 - Synkronisera ändringar



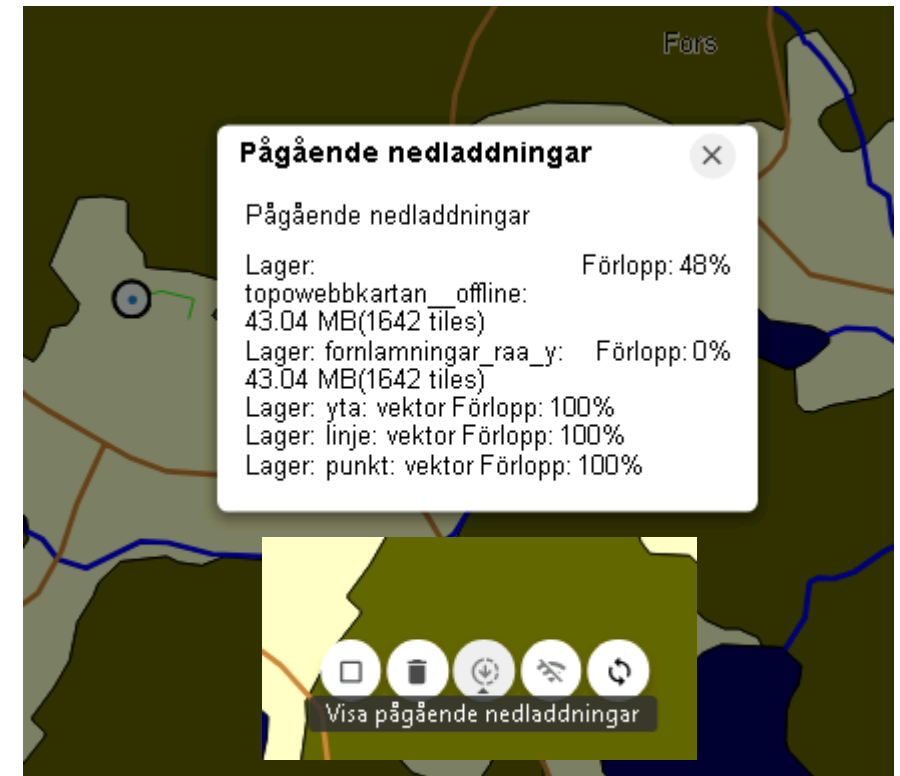
Ladda ner data lokalt

- Görs med fördel via WIFI innan man ger sig iväg för att spara surfpott
- Laddar ner högupplösta raster, t.ex. orto/topowebb inom vald rektangel
- Laddar ner vektorlager, antingen inom vald rektangel eller definierat extent beroende på konfiguration
 - Kan användas till vektorbakgrundskarta
- Man får en uppskattning om datamängd att ladda ner. OBS! Det drar snabbt iväg så kontrollera innan man startar.
- Man kan ladda ner flera områden. När man laddar ner ett nytt visas en ruta med var man redan laddat ner
- Mekanismen bakom detta är IndexedDb, vilket är en standard webbläsarteknologi. I praktiken sparas det på hårddisken och begränsningen i datamängd är knuten till tillgängligt utrymme på disken



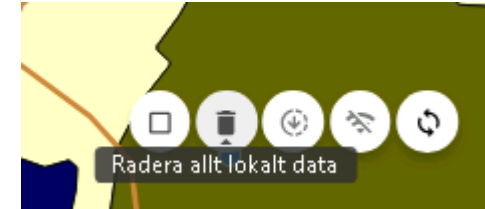
Visa pågående nedladdningar

- All nedladdning sker i bakgrunden så att man kan fortsätta jobba
- Via knappen ”visa pågående nedladdningar” får man upp en statusruta
- När allt är klart får man en toast



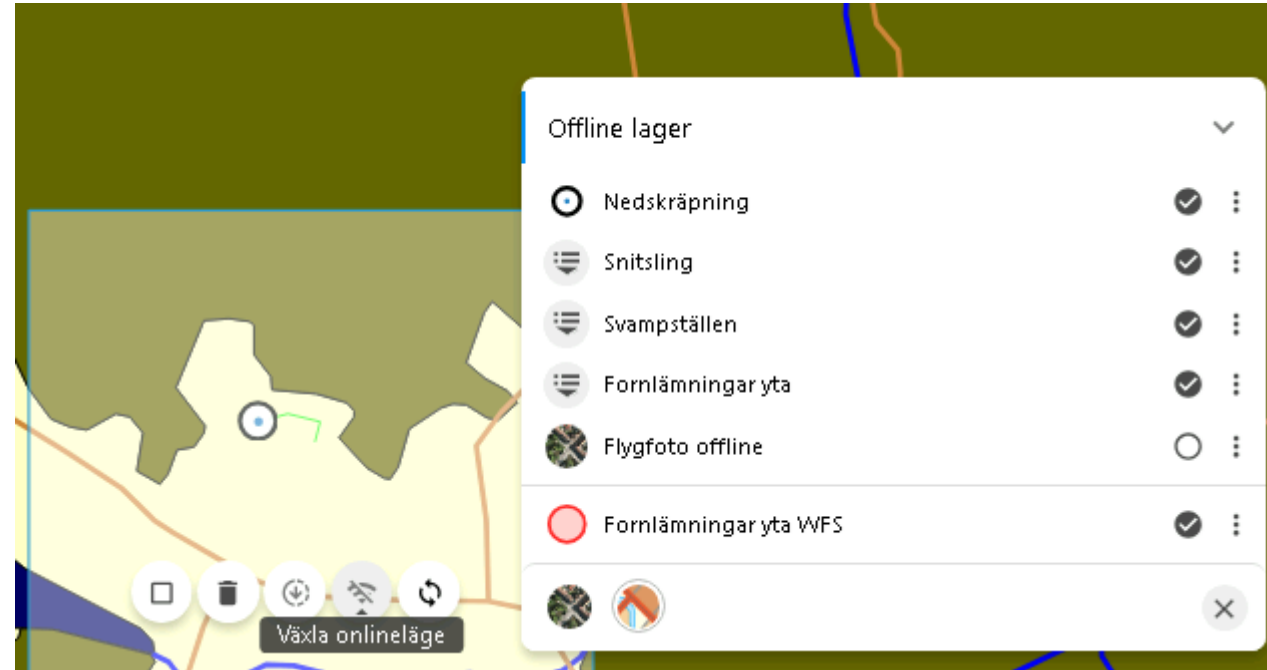
Radera lokalt data

- Raderar allt lokalt data
 - Tiles
 - Vektordata
 - Ej synkroniserade redigeringar
- Går inte att välja specifikt nedladdad område



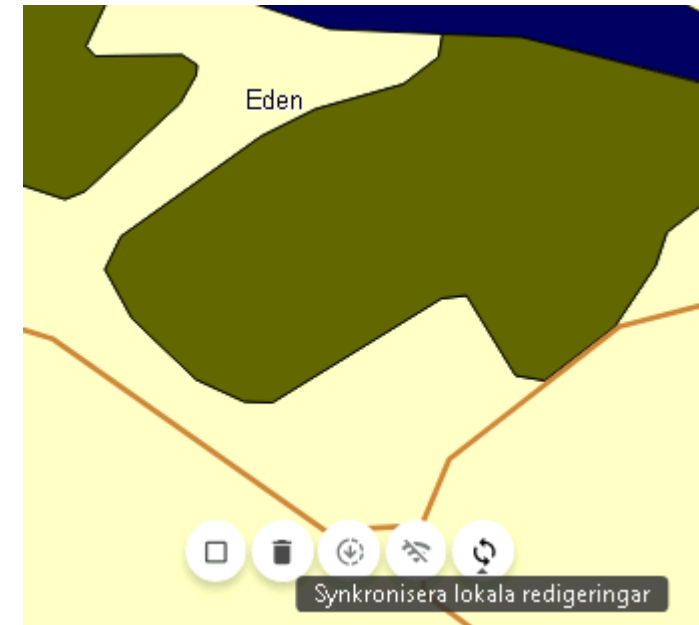
Offlineläget

- Gör egentligen inget annat än släcker och tänder lager
- Optimering för att slippa göra requests över långsamma uppkopplingar och undertrycka fel. Det är värre med en långsam uppkoppling än ingen alls
- Byter bakgrundskarta mellan en konfigurerad "offline"-bakgrund och den senast valda
- Släcker alla icke offline-kapabla lager och återställer dem vid återgång till online
 - Offlinekapabla är alla som är av typen lokala lager samt lager som explicit konfigurerats som offlinekapabla, typiskt de som ingår i ServiceWorkern
- Gör alltså ingenting för att ladda eller synkronisera lokala lager



Redigera offline

- Redigeringar görs precis som för ett vanligt vektorlager men när man sparar sparas ändringarna lokalt tillsammans med en ändringslogg i IndexedDb
- Via synkroniseringsknappen synkroniserar man lokala ändringar med servern via WFS-T
- Synkroniseringen har en enkel konflikthantering som aldrig skapar konflikter som kräver manuell interaktion
 - Först skickar man sina lokala ändringar till servern
 - Inserts kommer alltid in eftersom per definition är ett nytt objekt unikt i databasen
 - Updates kommer in om objektet finns kvar, dvs senast vinner (oavsett faktisk redigeringstid). Är den borta ur databasen ignoreras den (alltså inte senast vinner)
 - Deletes går in om objektet finns kvar, annars ignoreras den
 - Efter att ändringar applicerats på servern raderas allt lokalt data och laddas ner från databasen igen



Erfarenheter

- Det är lite krångligt med ServiceWorkers, speciellt vid utveckling
 - Webbbläsaren kör alltid en cachad version, har man gjort fel i ServiceWorker-koden i en publicerad version kanske inte den nya laddas ner
- Det är lite krångligt med behörigheter
 - För att applikationen skall bli användbar i fält vill man troligtvis ha den åtkomlig externt men man vill inte att den är publik, vilket kräver någon slags säkerhetslösning
 - Beroende på hur man löser det kan ServiceWorkern få problem att upptäcka nya versioner
 - Synkning mot WFS kan vara lite bökigt beroende på hur man löser inloggning
 - Vi har använt OpenID Connect i MobilityGuard, fast inte Origos OIDC-plugin då vi har en egen lösning som baseras på att varje Origo-applikation har en node.js server side
- Det finns ingen säkerhet i de lokala lagren eller IndexedDb
 - Kommer man in i telefonen kommer man åt data
 - Är vanligtvis inget problem, men man skall vara medveten om man delar enheten med andra eller saknar policy för skärmläs
- Det finns förutfattade meningar att övervinna kring begreppet "offline". Förväntningen är nog att det skall vara mer automagiskt.
- Man bör göra en dedikerad fältapplikation med minimal funktionalitet och en mer fullödlig adminkarta för kontorsjobb istället för att försöka få in allt i en och samma applikation för att minska strul i fält